

svmodt: An R Package for Linear SVM-Based Oblique Decision Trees

by Aneesh Agarwal, Jack Jewson, and Erik Sverdrup

Abstract Decision trees are widely used for classification tasks due to their interpretability, but traditional trees with axis-aligned splits often need to be deep in order to approximate complex decision boundaries. We first provide STreeR, the first fully native R implementation of the STree algorithm, validated against the original Python implementation. We then introduce svmodt, an R package implementing SVM-based oblique decision trees that use linear Support Vector Machines at each node to create multivariate splits. The package supports dynamic feature selection strategies, node-specific class weighting for handling imbalanced data, and feature diversity mechanisms through penalisation. We demonstrate that SVMODT outperforms axis-parallel decision trees while also maintaining shallower and more interpretable tree structures. The package includes comprehensive visualization tools, detailed documentation, and is freely available on GitHub.

1 Introduction

Decision trees remain widely used in machine learning because they are interpretable, straightforward to apply, and can accommodate both categorical and numerical predictors with minimal preprocessing. Classical algorithms such as Classification and Regression Trees (CART, (Breiman et al., 1984)) and C4.5 (Quinlan, 2014) employ axis-aligned (univariate) splits that partition the feature space along coordinate axes, thereby preserving transparency in the resulting decision rules. Consider a supervised classification task where we wish to predict $y \in \{1, \dots, K\}$, where K is the number of classes, from d -predictor variables $x \in \mathcal{X} \subseteq \mathbb{R}^d$. In this setting an axis parallel split constitutes selecting one feature $j \in \{1, \dots, d\}$ and a splitting value $\theta \in \mathbb{R}$ and splits according to the test $x_j \leq \theta$.

In R, traditional decision trees are readily implemented via packages such as `rpart`, `tree`, and `party`. However, the simple split structure of axis-parallel decision trees can promote overfitting, producing duplicated subtrees and repeated evaluations of the same predictor, which reduces efficiency and weakens generalisation (Pagallo and Haussler, 1990). Oblique decision trees mitigate these limitations by permitting splits on linear combinations of features; for example, a single oblique test of the form $\sum_{j=1}^d w_j x_j \leq \theta$, where $w = (w_1, \dots, w_d) \in \mathbb{R}^d$, can often replace multiple axis-aligned tests, producing more compact and geometrically simpler partitions of the feature space. Oblique trees are available in R through packages such as `ODRF`, `aorsf`, and `oblique.tree`. Empirical studies across diverse data sets show that oblique decision trees often achieve higher accuracy and yield more compact models than axis-parallel trees, though with a modest reduction in interpretability (Cañete et al., 2021).

Figure 1 contrasts an axis-parallel decision tree with an axis-oblique decision tree. The axis-parallel tree (left panel) fits an overly complex boundary, particular between the orange (circles) and purple (triangle) classes, and still has a high rate of training misclassification, reflecting its inability to capture interactions among predictors that determine class membership. In contrast, the oblique tree (right panel) yields geometrically simpler decision boundaries and substantially fewer misclassifications on the training set, indicating a better fit to the underlying multivariate relationships. Together, these panels illustrate how oblique splits can more effectively represent linear combinations of features and thereby reduce spurious complexity and error in tree-based classifiers.

A challenge with oblique decision trees, however, is deciding upon suitable oblique splits. The simplicity of axis-parallel trees allows algorithms such as CART (Breiman et al., 1984) to consider all possible splits, as for each j there is a maximum of $n - 1$ values of θ that would result in distinct splits. On the other hand, the added complexity of oblique increases the number of possible splits enormously, preventing exhaustive searches. Existing R packages address this by different means: `ODRF` uses random subspace projections, `aorsf` applies accelerated coordinate descent on the log-rank gradient, and `oblique.tree` employs linear discriminant functions as node classifiers.

Support Vector Machines (SVMs), introduced by Cortes and Vapnik (1995), provide a principled way to obtain oblique splits by finding margin-maximizing hyperplanes that separate the data in each node of the tree. This margin-based approach enhances the generalisation ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000), and allows for efficient estimation via quadratic programming. The idea of fitting SVMs at tree nodes was formalized by Bennett and Blue (1998) and extended to multiclass settings and other refinements in subsequent work (Takahashi and Abe, 2002; Bala and Agrawal, 2011; Ganaie et al., 2022). Python's STree implements SVM-node oblique trees using scikit-learn's SVC (Montañana et al., 2021; Pedregosa et al., 2011), while Java tools such as Weka offer components that can be adapted for hierarchical SVM trees (Menkovski

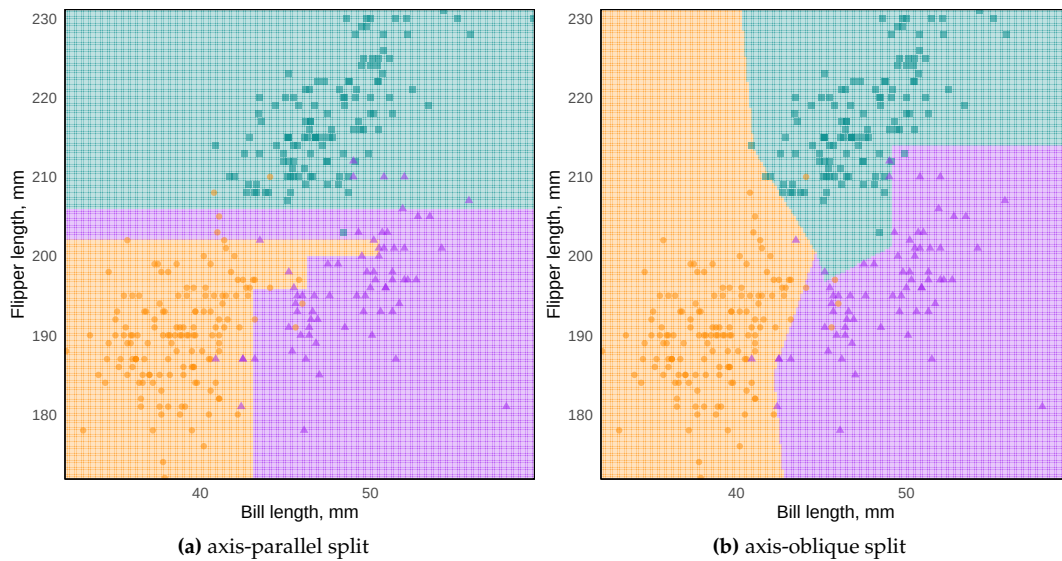


Figure 1: Comparison of Axis-parallel and Axis-oblique splits on Palmerpenguins dataset.

et al., 2008). However, a fully featured, native R implementation does not currently exist.

We address this gap through two primary contributions. First, we introduce a fully native R implementation of an STree-style support vector machine oblique decision tree that operates independently of Python. This implementation provides a production-ready interface and leverages vectorised computation to ensure computational efficiency and seamless integration within the R ecosystem. Second, we develop **SVM-ODT**, an enhanced support vector machine-based oblique decision tree that extends the original STree framework through several practical innovations. These include feature subsetting, which enables the selection of a subset of available features at each node; embedded feature selection, allowing features to be chosen based on predefined criteria such as mutual information or correlation, or through random selection; feature penalisation to discourage repeated use of the same features across splits; and node-specific class weighting to better handle class imbalance. Collectively, these enhancements are designed to improve predictive performance, interpretability, and robustness across both binary and multiclass classification settings. Together, these contributions aim to make the representational strengths of SVM-driven oblique splits accessible to R programmers while simultaneously addressing key computational and methodological limitations present in existing approaches.

2 Background

Decision Trees

Decision Trees (DTs) are interpretable classification models that represent their decision-making process through a hierarchical, tree-like structure. This structure comprises internal nodes containing splitting criteria that divide up the predictor space and terminal (leaf) nodes corresponding to predicted outcomes (class labels and probabilities). The nodes are connected by directed edges, each representing a possible True or False outcome of a splitting criterion for observations whose feature reach this node. Formally, a DT can be expressed as a rooted, directed tree $T = (G(V, E), v_1)$, where V denotes the set of nodes, E represents the set of edges linking these nodes, and v_1 is the root node.

Decision tree algorithms can be categorized based on whether the same type of split-test is applied at all internal nodes. **Homogeneous trees** employ a single algorithm throughout (e.g., univariate or multivariate splits), whereas **hybrid trees** allow different algorithms such as linear discriminant functions, k -nearest neighbors, or univariate splits that can be used in different sub-trees (Brodley and Utgoff, 1995). Hybrid trees exploit the principle of *selective superiority*, allowing subsets of the data to be modeled by the most appropriate classifier, thereby improving flexibility and accuracy. However, hybrid tree models typically incur substantial computational overhead due to their structural complexity, and therefore, in this paper we restrict our focus to homogeneous trees.

Axis-Parallel Decision Trees

Axis-Parallel Decision trees divide the feature space into several disjoint axis-parallel hypercubes. Axis-parallel decision trees, such as CART and C4.5, represent two of the most widely used algorithms for classification tasks. The CART algorithm employs a binary recursive partitioning procedure that is capable of handling both continuous and categorical variables as predictors or targets. At each node, the algorithm systematically evaluates every available variable $j \in \{1, \dots, d\}$ and all of its potential split points θ i.e. between any two observations, to determine the optimal partition. In R, traditional decision trees are readily implemented via packages such as `rpart`, `tree`, and `party`.

In contrast, **C4.5**, an extension of the earlier **ID3** algorithm (Quinlan, 1986), utilizing information theory measures such as **information gain** and **gain ratio** to select the most informative attribute for each split (Quinlan, 2014). C4.5 also includes mechanisms to handle missing attribute values by weighting instances according to the proportion of known data and employs an **error-based pruning** to eliminate subtrees that fail to improve predictive performance beyond a threshold (α), thereby mitigating overfitting. Although these techniques are effective across diverse data sets, studies have shown that the choice of pruning strategy and stopping criteria can significantly affect model performance across different domains (Mingers, 1989; Schaffer, 1992).

Axis-parallel decision trees offer strong interpretability but face inherent representational limits. Their constrained split structure often necessitates deep trees which can induce overfitting, leading to duplicated subtrees and repeated tests of the same predictor, ultimately reducing efficiency and weakening generalisation (Pagallo and Haussler, 1990).

Axis-Oblique Decision Trees

Axis-oblique decision trees extends axis-parallel decision trees by allowing each internal node to perform splits based on linear or nonlinear combinations (Chabbouh et al., 2025) of multiple features. This flexibility enables the tree to form oblique decision boundaries that more accurately partition the instance space. For example, a single multivariate test such as $x + y < 8$ can replace multiple univariate splits needed to approximate the same boundary. Oblique trees are available in R through packages such as `ODRF`, `aorsf`, and `oblique.tree`.

The construction of Axis-oblique decision trees introduces several design considerations, including how to represent multivariate tests, determine their coefficients, select features to include, handle symbolic and missing data, and prune to avoid over-fitting (Brodley and Utgoff, 1995). Various optimisation algorithms such as recursive least squares (Young, 1984), the pocket algorithm (Gallant, 1986), or thermal training (Frean, 1990) may be used to estimate the weights. However, Axis-oblique decision trees trade interpretability for representational power and often require additional mechanisms for **local feature selection**, such as *sequential forward selection* (SFS) or *sequential backward elimination* (SBE) (Kittler, 1986).

Although oblique splits introduce additional complexity and reduce interpretability, oblique decision trees retain key advantages of standard decision trees such as sequential split criteria evaluation and transparent decision procedures while offering improved modeling flexibility for complex data sets (Kozioł and Wozniak, 2009; Friedl and Brodley, 1997; Liu and Setiono, 1998).

Alternate Approaches to Oblique Decision Boundaries

Employing linear classifiers at internal nodes can yield decision trees that are both accurate and diverse. Because each node receives a different subset of the training data, the most suitable separating hyperplane may vary from node to node. For example, (Menze et al., 2011; Lemmond et al., 2010) use Linear Discriminant Analysis (LDA) to construct linear splits, though their approach is limited to binary classification. Zhang and Suganthan (2015) adopts Multisurface Proximal Support Vector Machine (MPSVM) to generate node-wise linear separators, while Truong (2009) fits multiple logistic-regression-based partitions (specifically $2^{K-1} - 1$ for K classes) and selects the split that best optimizes the impurity criterion.

Projection pursuit methods (Friedman and Tukey, 2006; Kruskal, 1969) have also been used to identify low-dimensional projections that expose structure in high-dimensional data. The **PPtree** (Lee et al., 2013; da Silva et al., 2021) algorithm extends this idea into a recursive partitioning framework by optimizing a class-sensitive projection index at each node and then applying standard split rules to the resulting one-dimensional projection. Similarly, Bulò and Kontschieder (2014) introduce a randomized multi-layer perceptron as a node-level splitting function. However, determining an appropriate network complexity remains challenging, as overly flexible models risk substantial overfitting.

Support Vector Machines (SVMs)

Support Vector Machines (SVMs) are powerful supervised learning models used predominantly for classification tasks, although extensions to regression do exist. They aim to determine an optimal separating hyperplane that maximizes the margin between binary classes in the data. This margin-based approach enhances the generalisation ability of the model, making SVMs robust and effective for many real-world problems (Cristianini, 2000).

The **linear SVMs** construct a separating hyperplane in the d -dimensional feature space such that the distance between the closest points of each class (the **support vectors**) and the hyperplane itself (Cortes and Vapnik, 1995). Given a training dataset $\{(x_i, y_i)\}_{i=1}^N$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$, the decision function is defined as $f(x) = \text{sign}(w^\top x + b)$. Training an SVM (estimating w) entails solving a convex quadratic programming (QP) problem involving an $(n \times n)$ kernel matrix ($K_{jk} = k(x_j, x_k) = x_j x_k^\top$). Owing to the convexity of the objective, the QP admits a unique global optimum and can be solved reliably and efficiently using standard optimisation techniques. While linear classifiers provide useful insights, they are often inadequate for real-world data sets, where classes are not linearly separable. In such cases, SVMs can be extended to create **nonlinear decision boundaries** by mapping the input vectors into a higher-dimensional **feature space** using a nonlinear transformation $\phi : \mathbb{R}^d \rightarrow \mathcal{F}$. So doing is facilitated computationally by the kernel trick which writes simplifies $K_{jk} = k(x_j, x_k) = \phi(x_j) \phi(x_k)^\top$.

Although SVMs exhibit strong theoretical foundations and robust generalisation capabilities, they present several practical limitations, most notably their lack of interpretability and effective methods for feature selection. Further, model performance is highly dependent on the appropriate selection of hyperparameters such as the regularisation term (C) and kernel parameters (e.g., γ), which govern the trade-off between margin maximization and misclassification tolerance (Nanda et al., 2018). For large-scale data sets, training an SVM can become computationally expensive, with both runtime and memory consumption growing quadratically in the number of training samples (Dong et al., 2005). Moreover, SVMs are inherently designed for binary classification, necessitating decomposition strategies such as One-vs-One and One-vs-All for multi-class problems (Hsu and Lin, 2002). Their performance also tends to degrade in imbalanced data settings, where the decision boundary becomes biased toward the majority class (Cervantes et al., 2020).

Oblique Decision Trees with Support Vector Machines

Support Vector Machine-based decision trees were first formalized by Bennett and Blue (1998), who adapted Statistical Learning Theory and Structural Risk Minimization to construct binary decision trees in which each internal node is an SVM that partitions the data by an optimal hyperplane. This formulation enabled multivariate, margin-based decisions at every split. Takahashi and Abe (2002) extended the idea to multiclass settings by using a recursive partitioning strategy: the root node isolates the most separable class or classes from the remainder, and the procedure recurses until each leaf contains a single class, thereby covering all classes. Subsequent studies have built upon this foundation to address scalability, optimisation, and generalisation issues. Optimal Decision Tree SVM (ODT-SVM) (Bala and Agrawal, 2011) introduced split-selection criteria based on Gini index, information gain, and scatter matrix separability to balance tree interpretability and margin-based precision. Reynolds et al. (2019) further extended the framework by incorporating class-specific cost parameters through the assignment of weights to individual classes, improving performance in imbalanced classification settings.

STree

More recent approaches embed multiple SVM classifiers directly within each split to handle multi-class problems more efficiently. STree (Montañana et al., 2021; Montañana et al., 2021) is an oblique multi-class decision tree algorithm that integrates support vector machines (SVMs) to construct single margin-based splits capable of handling multiclass problems. Unlike traditional multi-class tree methods that rely on clustering or ensembles of binary models, STree builds a single decision tree. Its central innovation lies in using SVM-derived hyperplanes at each internal node, enabling more expressive splits than axis-aligned trees while also being interpretable and computationally simple.

The algorithm generates a binary tree recursively (see Algorithm 5 in the Appendix). At each node:

- 1) Stopping conditions are evaluated to check the depth of the tree and the class purity. If the stopping conditions are met, a leaf node is created, labelled with the most frequent class in the node.

- 2) If the stopping conditions haven't been met the algorithm then selects the best hyperplane to split the data into two partitions: T^+ (positive side) and T^- (negative side).

When all instances at a node belong to more than two classes ($k' > 2$), the approach generates multiple candidate binary splits: For each class label y_i , it constructs a one-vs-rest binary problem: Class y_i vs. all other classes. It trains an SVM for each of these k' problems, resulting in k' hyperplanes $H_i, i = 1, \dots, K$. Each hyperplane partitions the data into T_i^+ and T_i^- . The algorithm computes the impurity (using Shannon entropy) of the class distribution within each partition. When only two classes remain, STree trains a single SVM to obtain the maximum-margin hyperplane. Instances are then routed left or right based on the sign of their distance to this hyperplane. STree's performance depends heavily on hyperparameter tuning, including kernel choice (linear, polynomial or Gaussian), gamma, polynomial degree, regularisation parameter C , and max iteration.

3 Contributed Methods

STreeR

A key contribution of this work is the first native R implementation of the STree algorithm. As no such implementation currently exists, we reconstruct the method in R using [e1071](#), which interfaces with the LIBSVM C++ backend. This reconstruction provides greater transparency into the algorithm's internal workflow and enables fair benchmarking against competing approaches.

The STree procedure constructs the decision tree recursively, where each internal node corresponds to a binary split induced by a support vector machine. At each node, a one-versus-rest strategy is employed: for each class y'_j , a binary SVM is trained to separate observations belonging to y'_j from all remaining classes. Training the SVM corresponds to estimating the optimal weight vector w_j^* that defines the separating hyperplane. The resulting decision rule assigns an observation x_i to the positive or negative partition based on the sign of the decision function, that is, $\text{sign}((w_j^*)^\top x_i)$.

Among all candidate one-versus-rest classifiers, the split that maximises information gain is selected. If the maximum information gain is positive, the dataset is partitioned accordingly and the procedure is applied recursively to each subset. Otherwise, a leaf node is created, and the majority class is assigned. This recursive splitting continues until a stopping criterion is satisfied, thereby yielding a complete oblique decision tree.

Algorithm 1 provides a concise summary of the recursive procedure underlying the STreeR implementation.

Table 1: Comparison of mean prediction accuracy and median training time for StreeR and STree. Here, N denotes the number of observations, X the number of features, and L the number of classes.

Dataset	N	X	L	StreeR	STree	StreeR(Med)	STree(Med)
WDBC Diagnosis	569	30	2	0.970 ± 0.004	0.970 ± 0.003	0.043ms	0.019ms
Iris	150	4	3	0.966 ± 0.006	0.965 ± 0.010	0.011ms	0.007ms
Echocardiogram	131	10	2	0.846 ± 0.004	0.845 ± 0.008	0.007ms	0.006ms
Fertility	100	9	2	0.876 ± 0.011	0.874 ± 0.010	0.003ms	0.005ms
Wine	178	13	3	0.978 ± 0.008	0.959 ± 0.008	0.023ms	0.007ms
Cardiotocography-3	2126	21	3	0.901 ± 0.003	0.903 ± 0.003	0.97ms	0.242ms
Cardiotocography-10	2126	21	10	0.792 ± 0.004	0.795 ± 0.018	4.391ms	0.474ms
Ionosphere	351	33	2	0.896 ± 0.009	0.892 ± 0.015	0.049ms	0.021ms
Dermatology	366	34	6	0.972 ± 0.006	0.959 ± 0.004	0.126ms	0.019ms

Algorithm 1: STreeR: Multi-class OVR SVM-based oblique decision tree

Input: $\mathcal{D}' = \{(x_i, y_i)\}_{i=1}^N$: Data
Output: *tree*: leaf node or data split $\mathcal{D}'^+, \mathcal{D}'^-$

```

1 function STree( $\mathcal{D}'$ ):
2   if stopping_condition( $\{y_i\}_{i=1}^t$ ) then
3     return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))           // Leaf node
4    $k' \leftarrow \text{num\_different\_labels}(\{y_i\}_{i=1}^t)$ 
5    $\mathcal{Y}' \leftarrow \{y'_1, \dots, y'_{k'}\}$                        // set of labels
6    $I_{\text{all}} \leftarrow I(\{y_i\}_{i=1}^t)$                        //  $I(\cdot)$  is an information theory measure
7    $n_{\text{models}} \leftarrow k'$                                // OVR
8   for  $j = 1$  to  $n_{\text{models}}$  do
9     if ( $j = k' = 2$ ) then
10      break                                             // Two labels, one SVM is enough
11     $\text{models}[j] \leftarrow \text{SVM}(\mathcal{D}')$  using binary class  $y'_j$  (+) vs rest (-) // OVR
12   $\mathcal{D}'^+, \mathcal{D}'^- \leftarrow \text{models}[j](\vec{x}_j) \quad \forall (\vec{x}) \in \mathcal{D}'$ 
13   $I_j \leftarrow \frac{|\mathcal{D}'^+|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}'^+|}) + \frac{|\mathcal{D}'^-|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}'^-|})$ 
14   $IG_j \leftarrow I_{\text{all}} - I_j$ 
15   $b^* = \arg \max_{j=1, \dots, n_{\text{models}}} IG_j$ 
16  if  $IG_{b^*} > 0$  then
17     $\text{node} \leftarrow \text{create\_node}(\text{models}[b^*])$ 
18     $\text{node.left} \leftarrow \text{STree}(\mathcal{D}'^+)$ 
19     $\text{node.right} \leftarrow \text{STree}(\mathcal{D}'^-)$ 
20  else
21    return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))           // No split gain
22  return node

```

To verify the accuracy of our R implementation of STree, we conducted a benchmarking comparison against the original Python version using a 5-fold cross-validation repeated 10 times with different seeds applied to 9 benchmark datasets from the UCI Machine Learning Repository. For comparability, both algorithms were run with the same default hyperparameters: cost-complexity $C = 1$, a linear kernel, a maximum of 1×10^7 training iterations, and a maximum depth of 10. Prediction accuracies were averaged across all folds for each dataset.

Across most datasets, StreeR exhibits performance that is effectively identical to its Python counterpart (see Table 1). Although both implementations rely on the same underlying LIBSVM C++ library, Python through the SVC classifier and R through the `e1071::svm` interface, there exist subtle yet meaningful differences in how each framework invokes the underlying routines responsible for fitting the support vector machine decision boundary and applying scaling transformations during prediction. These implementation-level discrepancies are likely to contribute to the small observed variation in performance.

To mitigate differences attributable to feature scaling, each dataset has been pre-processed in R using an implementation that replicates the behaviour of Python's standard scaling procedure.

Given computational and scope constraints, our re-implementation focuses only on the components of the STree algorithm that are directly relevant to our research objectives. In particular, we restrict our analysis to the one-vs-rest (OVR) splitting strategy. The alternative one-vs-one scheme is not included, as it scales poorly with the number of classes: for problems with large L , the number of pairwise SVMs grows quadratically, substantially increasing training time without offering clear benefits for the aspects of the algorithm we aim to study. We also implement only a linear kernel, both to maintain the interpretability of the resulting tree structure and to avoid the considerable computational overhead associated with non-linear kernels. At present, the implementation does not include hyperparameters for feature subsetting, although users can replicate this functionality by manually providing a reduced feature set.

Support Vector Machine based Oblique Decision Trees: SVMODT

A secondary contribution of this work is an extension of the STree framework to incorporate feature subsetting, dynamic feature selection, regularisation, feature penalisation, and class weighting, thereby introducing flexibility in model fitting. The resulting method, termed the Support Vector Machine-based Oblique Decision Tree (SVMODT), constructs a binary classification tree in which each internal node v at depth d is associated with a locally fitted support vector machine.

While several components, such as dynamic feature subsetting, are reconstructed from the original STree framework, we also refine existing mechanisms including feature subsetting and regularisation to provide greater adaptability. In addition, we introduce new controls for feature penalisation and node-specific class weighting. We further simplify the original STree structure by restricting node classifiers to linear kernels, thereby removing the need for multiple kernel-specific hyperparameters and improving interpretability, computational efficiency, and ease of inference.

At each node, the algorithm (see Algorithm 4), implemented in R using the `e1071` interface to LIBSVM, performs the following operations. A subset of features is first selected according to a specified strategy, which may remain constant, decrease with tree depth, or be randomly sampled within a predefined range. Within this subset, features are then prioritised using a criterion such as mutual information, correlation, or random assignment. To encourage diversity across splits, a penalisation mechanism reduces the likelihood of repeatedly selecting features that have been used in previous nodes.

Given the selected feature set, a binary support vector machine is trained at node v , optionally incorporating class-specific weights to account for imbalance in the local data distribution. The fitted classifier defines a separating hyperplane characterised by a weight vector w_v^* , and observations are partitioned according to the sign of the decision function $\text{sign}((w_v^*)^\top x)$. The resulting split is evaluated using an impurity-based criterion, and the procedure is recursively applied to the resulting child nodes until a stopping condition is satisfied.

Collectively, these extensions enable SVMODT to adapt both its feature space and decision boundaries dynamically throughout the tree, thereby improving predictive performance and robustness, particularly in high-dimensional and multiclass settings. We now discuss the key features of SVMODT and how these differentiate it from the original STree framework.

Key Features

Binary & Multiclass SVM Training At each internal node of the tree, a Support Vector Machine (SVM) is trained to construct an oblique decision boundary that partitions the data into two child nodes. The training procedure differs depending on whether the node contains samples from two classes or more than two classes.

Binary Classification When the node contains exactly two classes ($k = 2$), a binary SVM classifier is trained (see ALGORITHM 2) using the selected feature subset $\mathcal{F}_d$. Let $\mathbf{X}_{\text{scaled}}$ denote the standardized feature matrix at the node and $\mathcal{Y}$ the corresponding class labels.

Algorithm 2: Binary Classification Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w : class weighting strategy
Output: (model, $\mathcal{I}_L, \mathcal{I}_R$): SVM model, left indices, right indices, split class

```

1 function BinarySplit( $\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w$ ):
2    $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}, \text{strat}_w)$ 
3    $\text{model} \leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, w_c)$ 
4   if  $\text{model} = \text{null}$  then
5     return (null,  $\emptyset, \emptyset$ , null)
6    $\mathbf{f} \leftarrow \text{model.decision\_values}(\mathbf{X}_{\text{scaled}})$ 
7    $\mathcal{I}_L \leftarrow \{i : f_i > 0\}, \quad \mathcal{I}_R \leftarrow \{i : f_i \leq 0\}$ 
8   return (model,  $\mathcal{I}_L, \mathcal{I}_R$ , null)

```

The SVM learns a separating hyperplane of the form:

$$\begin{aligned}
 & \arg \min_{\beta \in \mathbb{R}^{d+1}, \xi \in \mathbb{R}_+^N} \frac{1}{2} \beta_{1:d}^\top \beta_{1:d} + C \sum_{n=1}^N \xi_n \\
 & \text{subject to } y_n (\beta^\top \mathbf{x}_n) \geq 1 - \xi_n, \quad n = 1, \dots, N, \\
 & \quad \xi_n \geq 0, \quad n = 1, \dots, N.
 \end{aligned}$$

where β represents the weight vector and ξ_n are slack variables that permit margin violations. The hyperplane partitions the data into two subsets according to the value of the decision function. Samples satisfying $f(x) \leq 0$ are assigned to the left child node, while those with $f(x) > 0$ are assigned to the right child node. Class weights may optionally be applied during SVM training to address class imbalance. The resulting partition indices $\mathcal{I}_L$ and $\mathcal{I}_R$ are then used to recursively construct the left and right subtrees.

Multiclass Classification When more than two classes are present at the node ($k > 2$), a multi-class SVM strategy is employed (see Algorithm 3) to generate a binary split. The algorithm evaluates candidate partitions of the class set in order to determine a split that maximizes impurity reduction.

Algorithm 3: Multiclass One-vs-Rest Split

Input: $\mathbf{X}_{\text{scaled}}$: scaled features, $\mathcal{Y}$: labels, strat_w : class weighting strategy, I_{parent} : parent impurity, measure_I : impurity measure, τ_I : minimum information gain, n : number of samples

Output: (best_model, $\mathcal{I}_L, \mathcal{I}_R$): best SVM model, left indices, right indices, split class

```

1 function MulticlassSplit( $\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w, I_{\text{parent}}, \text{measure}_I, \tau_I, n$ ):
2    $k \leftarrow |\text{unique}(\mathcal{Y})|$ 
3    $n_{\text{models}} \leftarrow k$ 
4   for  $j = 1$  to  $n_{\text{models}}$  do
5      $c_j \leftarrow \text{unique}(\mathcal{Y})[j]$  // Target class for OvR
6      $\mathcal{Y}_{\text{binary}} \leftarrow \begin{cases} + & \text{if } y_i = c_j \\ - & \text{otherwise} \end{cases}$ 
7      $w_c \leftarrow \text{calculate\_class\_weights}(\mathcal{Y}_{\text{binary}}, \text{strat}_w)$ 
8      $\text{model}_j \leftarrow \text{fit\_linear\_SVM}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}_{\text{binary}}, w_c)$ 
9     if  $\text{model}_j = \text{null}$  then
10      continue // Skip failed SVM
11      $\mathbf{f}_j \leftarrow \text{model}_j.\text{decision\_values}(\mathbf{X}_{\text{scaled}})$ 
12      $\mathcal{I}_L^j \leftarrow \{i : f_{j,i} > 0\}, \quad \mathcal{I}_R^j \leftarrow \{i : f_{j,i} \leq 0\}$ 
13     if  $|\mathcal{I}_L^j| = 0$  or  $|\mathcal{I}_R^j| = 0$  then
14      continue // Skip degenerate split
15      $I_L^j \leftarrow I(\mathcal{Y}[\mathcal{I}_L^j], \text{measure}_I)$ 
16      $I_R^j \leftarrow I(\mathcal{Y}[\mathcal{I}_R^j], \text{measure}_I)$ 
17      $I_j \leftarrow \frac{|\mathcal{I}_L^j|}{n} \cdot I_L^j + \frac{|\mathcal{I}_R^j|}{n} \cdot I_R^j$ 
18      $IG_j \leftarrow I_{\text{parent}} - I_j$ 
19     if  $IG_j < \tau_I$  then
20      continue // Insufficient information gain
21    $b^* \leftarrow \arg \min_j I_j$  // Select best split
22   if  $\text{model}_{b^*} = \text{null}$  then
23     return (null,  $\emptyset, \emptyset, \text{null}$ )
24   return ( $\text{model}_{b^*}, \mathcal{I}_L^{b^*}, \mathcal{I}_R^{b^*}$ )

```

For each candidate split, the classes are divided into two groups, and a binary SVM is trained to separate the corresponding subsets of samples. The quality of the resulting partition is evaluated using an impurity reduction criterion such as information gain. Let $I(\mathcal{Y}_{\sqrt{\cdot}})$ denote the impurity of the parent node and $I(\mathcal{Y}_L)$ and $I(\mathcal{Y}_R)$ denote the impurities of the resulting child nodes. The information gain is computed as

$$\Delta I = I(\mathcal{Y}_{\sqrt{\cdot}}) - \left(\frac{|\mathcal{I}_L|}{n_p} I(\mathcal{Y}_L) + \frac{|\mathcal{I}_R|}{n_p} I(\mathcal{Y}_R) \right),$$

where n_p is the number of samples at the node. The candidate split that yields the highest impurity reduction above a predefined threshold τ_I is selected as the optimal split. If no candidate partition produces a valid improvement or if one of the resulting child nodes is empty, the node is converted into a leaf node.

Maximum Feature Selection To control model complexity and encourage feature diversity across tree nodes, the algorithm dynamically limits the number of candidate features considered during split generation. Let p denote the total number of available predictors. At depth d of the tree, the maximum number of features evaluated is denoted by m_d . The value of m_d is determined using one of three configurable strategies:

$$m_d = \begin{cases} m_{\text{base}} & \text{constant strategy} \\ \lfloor m_{\text{base}} \cdot \alpha^{d-1} \rfloor & \text{decreasing strategy} \\ \text{Uniform}(\lfloor p \cdot \ell_{\min} \rfloor, \lfloor p \cdot \ell_{\max} \rfloor) & \text{random strategy} \end{cases}$$

In the **constant strategy**, the same number of candidate features m_{base} is evaluated at every depth of the tree. This provides consistent computational cost across nodes.

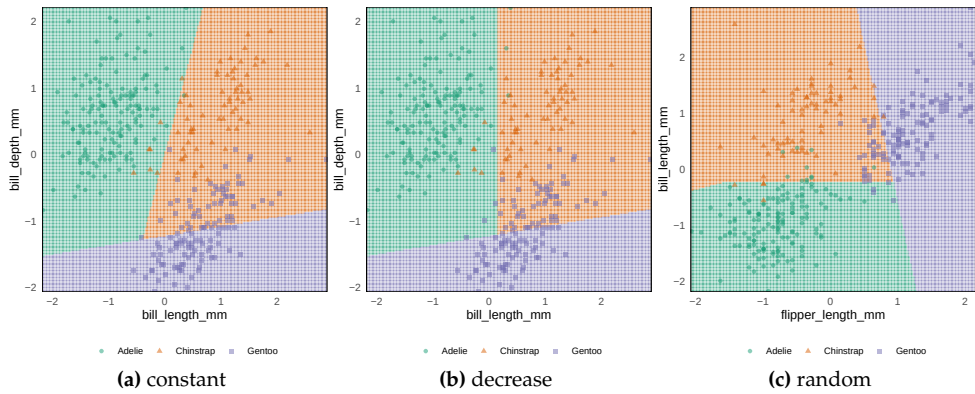


Figure 2: Comparison of constant, decreasing, and random maximum feature selection strategies using SVMODT on the Palmer Penguins dataset.

Table 2: Comparison of mean prediction accuracy (mean $\pm$ standard deviation) across maximum feature selection strategies for SVMODT, with results benchmarked against the Python STree implementation. All SVMODT models were trained using `max_depth = 15`, mutual-information feature ranking, and adaptive feature selection. The constant strategy uses a fixed number of candidate features at each node; the decrease strategy progressively reduces the number of available features with tree depth (decrease rate = 0.5); the random strategy samples the candidate feature proportion uniformly from $c(0.5, 0.8)$ at each split. Bold values indicate the highest prediction accuracy for each dataset.

Dataset	N	X	L	STree	Constant	Decrease	Random
WDBC Diagnosis	569	30	2	0.970 $\pm$ 0.003	0.969 $\pm$ 0.003	0.970 $\pm$ 0.004	0.967 $\pm$ 0.008
Iris	150	4	3	0.965 $\pm$ 0.010	0.966 $\pm$ 0.006	0.955 $\pm$ 0.003	0.963 $\pm$ 0.006
Echocardiogram	131	10	2	0.845 $\pm$ 0.008	0.846 $\pm$ 0.006	0.846 $\pm$ 0.004	0.850 $\pm$ 0.005
Fertility	100	9	2	0.874 $\pm$ 0.010	0.874 $\pm$ 0.010	0.875 $\pm$ 0.011	0.881 $\pm$ 0.000
Wine	178	13	3	0.959 $\pm$ 0.008	0.978 $\pm$ 0.008	0.976 $\pm$ 0.006	0.959 $\pm$ 0.009
Cardiotocography-3	2126	21	3	0.903 $\pm$ 0.003	0.902 $\pm$ 0.004	0.890 $\pm$ 0.002	0.906 $\pm$ 0.005
Cardiotocography-10	2126	21	10	0.795 $\pm$ 0.018	0.792 $\pm$ 0.004	0.630 $\pm$ 0.007	0.797 $\pm$ 0.006
Ionosphere	351	33	2	0.892 $\pm$ 0.015	0.896 $\pm$ 0.011	0.892 $\pm$ 0.012	0.884 $\pm$ 0.013
Dermatology	366	34	6	0.959 $\pm$ 0.004	0.972 $\pm$ 0.006	0.979 $\pm$ 0.006	0.971 $\pm$ 0.005

In the **decreasing strategy**, the number of candidate features gradually decreases as the tree depth increases. The parameter $\alpha \in (0, 1]$ controls the rate of reduction. This approach allows early splits to explore a larger feature space while later splits focus on a smaller set of predictors.

In the **random strategy**, the number of candidate features is sampled from a uniform distribution bounded by fractions $\ell_{\min}$ and $\ell_{\max}$ of the total feature count p . This introduces additional stochasticity into the tree construction process and may improve generalisation by reducing overfitting.

Once m_d is determined, up to m_d features are selected using the specified feature selection method. The resulting subset is then used to train the SVM model responsible for generating the oblique split at the node.

Figure 2 displays the decision surface of the fitted SVMODT model under alternative specifications of the maximum feature selection strategy (see the HTML version for the multidimensional tour-based visualisation). The coloured regions represent the predicted class labels of the fitted tree evaluated over a grid of values, with the observed data points overlaid for reference. The first two selected features define the horizontal and vertical axes, while all remaining features are fixed at their median values.

The figure illustrates how the maximum feature selection strategy influences the geometry of the induced decision boundaries. Under the constant strategy, the model retains the same number of candidate variables at each node, resulting in more pronounced oblique splits across the two plotted dimensions. Under the decreasing strategy, the number of available features declines with depth, and the separation between Adelle and Chinstrap becomes increasingly axis-parallel. This suggests that information loss in deeper nodes restricts the model’s ability to construct fully oblique partitions. Under the random strategy, the subset of candidate variables varies stochastically across nodes, producing some variation in the selected predictors. Nevertheless, the overall decision surface remains broadly similar to the previous two strategies, indicating a degree of structural robustness in the fitted tree.

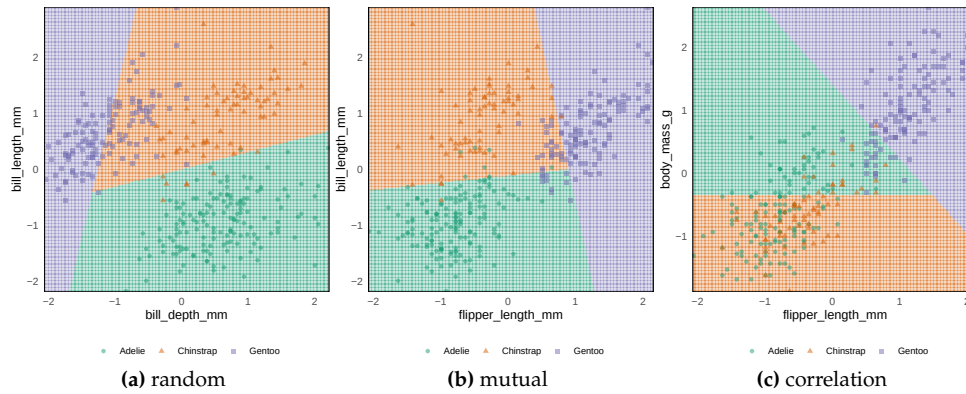


Figure 3: Comparison of random, mutual, and correlated feature selection methods using SVMODT on the Palmer Penguins dataset.

Table 2 summarizes SVMODT performance across different maximum feature selection strategies compared to the Python STree benchmark. Overall, predictive accuracy is largely consistent across strategies, with differences generally within a few percentage points. For most datasets, all three maximum feature strategies; Constant, Decrease, and Random perform similarly to the STree benchmark, demonstrating the robustness of SVMODT to variations in maximum feature selection. Some datasets, such as Wine, Cardiotocography-10, and Dermatology, exhibit notable differences. For example, the Decrease strategy underperforms on Cardiotocography-10 relative to Constant and Random, while Random selection yields the highest accuracy for Fertility. In datasets with smaller feature sets or fewer classes, differences between strategies are minimal, indicating that the choice of maximum feature strategy has limited impact when feature dimensionality is low. These results suggest that SVMODT is generally stable across maximum feature selection strategies, but careful selection of the strategy may yield marginal gains on larger or more complex datasets.

Feature Selection Method At each node of the tree, a subset of candidate features is selected from the full feature set $\mathcal{F}$ to construct the SVM-based oblique split. This process encourages diversity across nodes in the tree. Let $p = |\mathcal{F}|$ denote the total number of available features. At depth d , the algorithm first determines the maximum number of features m_d that may be considered for splitting. This value is computed according to the selected maximum-feature strategy. A subset of up to m_d features is then chosen using a specified feature selection method.

Figure 3 illustrates the effect of alternative feature selection strategies on the splits generated by SVMODT. The resulting decision surfaces differ substantially across specifications, demonstrating the flexibility of the proposed framework and the sensitivity of tree structure to the feature selection mechanism employed at each node. Under the random strategy, the model selects `bill_depth_mm` and `bill_length_mm`, yielding boundaries based on stochastic feature choice. The mutual information strategy selects variables with the strongest dependence on the response, namely `flipper_length_mm` and `bill_length_mm`, producing partitions aligned with the most informative predictors. In contrast, the correlation-based strategy selects `flipper_length_mm` and `body_mass_g`, reflecting variables most strongly linearly associated with the class labels. Collectively, these results highlight how alternative feature selection criteria induce distinct geometric partitions while preserving classification performance.

The package supports multiple feature selection approaches:

- **Random Selection:** A subset of features is sampled uniformly at random from the available feature set. This strategy introduces stochasticity into the tree construction process and helps reduce overfitting.
- **Mutual Information Ranking:** Features are ranked based on their mutual information with the target variable. Mutual information measures the statistical dependency between a feature and the class labels. Mutual information quantifies the reduction in uncertainty about the class label obtained by observing a given feature, thereby measuring the statistical dependency between predictors and the response. Features with higher mutual information are considered more informative for classification, and the top-ranked variables are selected for model training. This ranking is implemented using the `FSelectorRcpp` package.
- **Correlation-Based Filtering:** Features are ranked using a univariate association measure between each predictor and the response variable. Predictors with larger association values

Table 3: Comparison of mean prediction accuracy across feature selection strategies for SVMODT, benchmarked against the Python STree implementation. All SVMODT models were trained using $\text{max_depth} = 15$. The random strategy selects candidate predictors using random subset sampling ($n_subsets = 10$); the mutual information strategy ranks predictors according to their dependency with the response variable; the correlation strategy ranks predictors using absolute correlation with the response. STree does not employ an explicit pre-split feature ranking mechanism, instead optimising the node classifier directly on the available predictors. Bold values indicate the highest prediction accuracy for each dataset.

Dataset	N	X	L	STree (Benchmark)	Random	Mutual Info	Correlation
WDBC Diagnosis	569	30	2	0.970 $\pm$ 0.003	0.967 $\pm$ 0.003	0.968 $\pm$ 0.006	0.970 $\pm$ 0.004
Iris	150	4	3	0.965 $\pm$ 0.010	0.955 $\pm$ 0.010	0.960 $\pm$ 0.009	0.961 $\pm$ 0.008
Echocardiogram	131	10	2	0.845 $\pm$ 0.008	0.848 $\pm$ 0.005	0.851 $\pm$ 0.007	0.847 $\pm$ 0.007
Fertility	100	9	2	0.874 $\pm$ 0.010	0.880 $\pm$ 0.003	0.878 $\pm$ 0.009	0.878 $\pm$ 0.009
Wine	178	13	3	0.959 $\pm$ 0.008	0.962 $\pm$ 0.016	0.962 $\pm$ 0.016	0.961 $\pm$ 0.012
Cardiotocography-3	2126	21	3	0.903 $\pm$ 0.003	0.900 $\pm$ 0.007	0.903 $\pm$ 0.002	0.899 $\pm$ 0.005
Cardiotocography-10	2126	21	10	0.795 $\pm$ 0.018	0.790 $\pm$ 0.008	0.798 $\pm$ 0.006	0.793 $\pm$ 0.007
Ionosphere	351	33	2	0.892 $\pm$ 0.015	0.889 $\pm$ 0.020	0.888 $\pm$ 0.010	0.881 $\pm$ 0.012
Dermatology	366	34	6	0.959 $\pm$ 0.004	0.958 $\pm$ 0.011	0.969 $\pm$ 0.007	0.966 $\pm$ 0.003

are considered more relevant, and the highest-ranked features are retained for model fitting. This approach provides a computationally efficient screening rule that prioritises individually informative variables while reducing the influence of weak or irrelevant predictors.

To further encourage diversity across tree nodes, the algorithm incorporates a feature reuse penalty mechanism. Previously selected features are tracked in the set $\mathcal{F}_{\text{used}}$, and a penalty parameter λ_{pen} reduces the probability that these features are repeatedly chosen at deeper levels of the tree.

After the candidate feature subset $\mathcal{F}_d$ is selected, the corresponding predictors are standardized at the node level. Features that become constant after preprocessing are removed. If no valid features remain, the node is converted into a leaf node. The selected feature subset is then used to train the SVM model responsible for generating the oblique decision boundary at the node.

For strategies involving random feature subsets, multiple candidate subsets may be evaluated. For each candidate subset, a Support Vector Machine is fitted and the resulting split is assessed using an impurity based criterion such as information gain. The subset that produces the highest impurity reduction is selected for the node. This approach enables the tree to identify informative oblique decision boundaries while maintaining computational tractability.

Table 3 compares classification performance of different feature selection strategies evaluated across ten benchmark datasets. Across the datasets, the results indicate that all feature selection strategies achieve competitive performance, with relatively small differences in accuracy. This suggests that the SVM based oblique splitting mechanism remains robust to the specific feature selection approach used at each node. Random feature selection performs consistently well and achieves accuracy values that are comparable to, and occasionally exceed, the benchmark model. Mutual Information and Correlation based methods provide slightly more stable performance on certain datasets where informative predictors might exhibit stronger statistical relationships with the target variable.

Feature Penalty Mechanism To encourage feature diversity across tree nodes and reduce repeated reliance on the same predictors, SVMODT incorporates a feature penalisation mechanism during feature selection. Features that have already been selected in earlier splits receive a reduced relevance score when candidate variables are evaluated at deeper nodes. This encourages the tree to explore alternative predictors rather than repeatedly relying on a small subset of dominant variables.

Figure 4 illustrates the effect of varying the penalisation strength on the fitted SVMODT model. Separate trees are estimated under different penalty values, after which the number of distinct features used and their selection frequencies are recorded. As the penalty increases, the algorithm is increasingly encouraged to utilise a broader set of predictors, thereby promoting structural diversity within the tree.

Let $\mathcal{F}_{\text{used}}$ denote the set of features previously selected in earlier nodes. Suppose s_j represents the original relevance score of feature j , such as its mutual information with the response variable. The penalised score, denoted $\tilde{s}_j$, is then defined as

$$\tilde{s}_j = \begin{cases} s_j, & \text{if } j \notin \mathcal{F}_{\text{used}}, \\ (1 - \lambda_{\text{pen}})s_j, & \text{if } j \in \mathcal{F}_{\text{used}}, \end{cases}$$

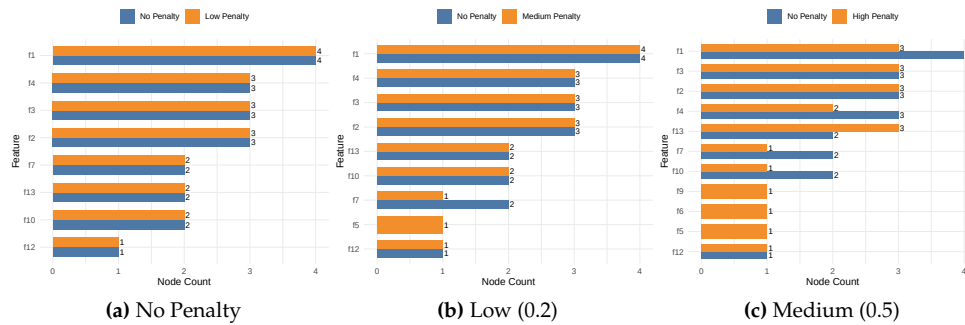


Figure 4: Comparison of Low, Medium and High penalty on repeated features by SVMODT on Wine dataset.

Table 4: Comparison of mean prediction accuracy (mean $\pm$ standard deviation) across feature penalisation strategies for SVMODT. All models were trained using `max_depth = 15`, mutual-information feature ranking, and adaptive maximum feature selection. No penalisation corresponds to reuse of previously selected features being unrestricted. Low, medium, and high penalisation correspond to feature penalty weights of 0.2, 0.5, and 0.8, respectively. Bold values indicate the highest accuracy for each dataset.

Dataset	N	X	L	None (Benchmark)	Low	Medium	High
WDBC Diagnosis	569	30	2	0.946 $\pm$ 0.005	0.950 $\pm$ 0.004	0.949 $\pm$ 0.007	0.949 $\pm$ 0.007
Iris	150	4	3	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006	0.966 $\pm$ 0.006
Echocardiogram	131	10	2	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001	0.855 $\pm$ 0.001
Fertility	100	9	2	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000	0.881 $\pm$ 0.000
Wine	178	13	3	0.950 $\pm$ 0.015	0.948 $\pm$ 0.016	0.951 $\pm$ 0.013	0.947 $\pm$ 0.011
Cardiotocography-3	2126	21	3	0.883 $\pm$ 0.006	0.883 $\pm$ 0.004	0.882 $\pm$ 0.003	0.880 $\pm$ 0.004
Cardiotocography-10	2126	21	10	0.782 $\pm$ 0.007	0.775 $\pm$ 0.008	0.776 $\pm$ 0.010	0.739 $\pm$ 0.034
Ionosphere	351	33	2	0.869 $\pm$ 0.018	0.868 $\pm$ 0.016	0.868 $\pm$ 0.017	0.868 $\pm$ 0.017
Dermatology	366	34	6	0.974 $\pm$ 0.006	0.977 $\pm$ 0.006	0.978 $\pm$ 0.006	0.968 $\pm$ 0.009

where $\lambda_{\text{pen}} \in [0, 1]$ controls the strength of the penalty. A value of $\lambda_{\text{pen}} = 0$ implies no penalisation, whereas larger values increasingly discourage the reuse of previously selected variables.

After selecting the feature subset $\mathcal{F}_d$ at depth d , the set of previously used features is updated according to

$$\mathcal{F}'_{\text{used}} = \mathcal{F}_{\text{used}} \cup \mathcal{F}_d.$$

This mechanism promotes exploration of alternative predictors throughout recursive tree construction, which may improve generalisation performance and reduce overfitting arising from repeated use of the same highly dominant features.

Table 4 compares SVMODT performance under different feature penalisation strategies compared to the benchmark (None). Overall, the impact of penalising previously used features varies across datasets. On datasets such as WDBC Diagnosis, Wine, and Dermatology, applying low or medium feature penalisation yields slightly higher prediction accuracy than the unpenalised benchmark. For example, Wine achieves the highest accuracy under Medium penalisation (0.951 $\pm$ 0.013). Several datasets, including Iris, Echocardiogram, and Fertility, show identical performance across all penalisation levels, suggesting that feature reuse does not influence accuracy in these cases. In datasets with more complex class structures or imbalanced classes, such as Cardiotocography-10, high feature penalisation reduces predictive performance (0.739 $\pm$ 0.034), indicating that excessive penalisation can hinder model learning. In summary, moderate feature penalisation can provide marginal improvements in predictive accuracy, while excessive penalisation may be detrimental, particularly for datasets with large numbers of classes or complex patterns. Feature penalisation strategies should therefore be tuned according to dataset characteristics to optimise SVMODT performance.

Regularisation Regularisation in SVMODT operates at both the node level and the global tree level to control model complexity, reduce overfitting, and improve generalisation. Unlike conventional decision trees that rely solely on pruning after tree construction, SVMODT incorporates pre-split regularisation mechanisms that govern whether additional partitions should be created during training.

A primary control is the minimum node size parameter, denoted min_n , which specifies the mini-

Table 5: Comparison of tree structure and predictive accuracy across minimum sample thresholds on the WDBC dataset. All SVMODT models were trained using the random maximum feature strategy and mutual information feature ranking. Higher minimum sample thresholds restrict further node splitting and therefore produce shallower tree structures.

Min Samples	Splits	Leaves	Total Nodes	Accuracy
10	3	4	7	0.9739
50	2	3	5	0.9913

Table 6: Comparison of tree structure and accuracy across different maximum tree depths on the Cardiotocography dataset.

Max Depth	Splits	Leaves	Total Nodes	Accuracy
3	6	7	13	0.8826
5	12	13	25	0.8991
7	14	15	29	0.8991

minimum number of observations required for a node to be eligible for further splitting. If the number of observations at node v falls below this threshold, the node is converted into a terminal leaf. This prevents unstable support vector machine fits in very small partitions and reduces the likelihood of learning decision boundaries driven by sampling noise. From Table @ref(tab:min_split-table-latex), it is evident that increasing the minimum sample threshold, min_n , generally produces shallower tree structures by restricting further splits in smaller nodes. This reduction in tree complexity can mitigate overfitting and, in several cases, improve out-of-sample predictive accuracy.

A second source of regularisation is the maximum tree depth parameter, d_{max} , which limits the number of recursive splits from the root node to any terminal node. Shallow trees produce simpler and more interpretable decision rules, whereas deeper trees allow greater flexibility and more localised decision boundaries. Constraining depth therefore provides a direct mechanism for balancing interpretability against predictive complexity.

Table 6 compares tree structure and predictive accuracy across different values of the maximum tree depth parameter. As expected, increasing max_depth permits additional recursive splits, resulting in larger trees with more leaves and internal nodes. Expanding the depth from 3 to 5 nearly doubles the number of splits and increases accuracy from 0.8826 to 0.8991, indicating that the shallower tree underfits the data. However, further increasing the depth from 5 to 7 produces only marginal structural growth, while predictive accuracy remains unchanged at 0.8991. This suggests that a depth of 5 is sufficient to capture the underlying decision structure, and deeper trees provide no additional generalisation benefit. Consequently, d_{max} acts as an effective regularisation parameter, where moderate values balance model flexibility and predictive performance.

A third mechanism is the minimum information gain threshold, implemented through an impurity improvement criterion. At each node, the candidate split induced by the support vector machine is accepted only if the resulting reduction in class impurity exceeds a predefined threshold (τ_I). If the improvement in impurity is negligible, the node is terminated rather than expanded further. This avoids unnecessary partitions that offer little predictive value and acts as a structural penalty on overly complex trees.

Table 7 reports the effect of varying the minimum impurity threshold on tree complexity and predictive accuracy for the Cardiotocography dataset. Increasing the threshold imposes a stricter requirement for accepting new splits, thereby reducing overall tree size. Raising the threshold from 0.005 to 0.01 decreases the number of splits from 12 to 10 and total nodes from 25 to 21, while accuracy remains unchanged at 0.8991. This indicates that several low-gain splits can be removed without loss of predictive performance. However, a substantially larger threshold of 0.1 further contracts the tree to only 6 splits and 13 total nodes, accompanied by a decline in accuracy to 0.8826. This suggests that excessive regularisation prevents the model from capturing important class structure. Overall, the minimum impurity threshold serves as an effective pruning mechanism, where moderate values simplify the tree without sacrificing accuracy, whereas overly restrictive values may induce underfitting.

Together, min_n , d_{max} and τ_I the information gain threshold form a complementary regularisation framework. The minimum node size controls local sample adequacy, maximum depth constrains global tree growth, and information gain ensures that each additional split contributes meaningful class separation. Their joint use enables SVMODT to maintain predictive flexibility while limiting overfitting, particularly in small, noisy, or high-dimensional datasets.

Class Weight Handling To address class imbalance, the algorithm allows three weighting schemes:

Table 7: Comparison of tree structure and accuracy across different minimum impurity threshold values on the Cardiotocography dataset.

Impurity Threshold	Splits	Leaves	Total Nodes	Accuracy
0.005	12	13	25	0.8991
0.01	10	11	21	0.8991
0.1	6	7	13	0.8826

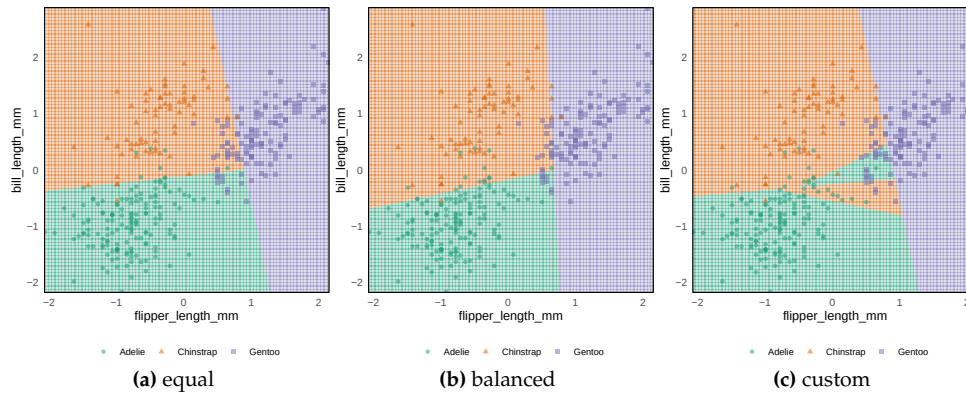


Figure 5: Comparison of equal, balanced, and custom weighting schemes using SVMODT on the Palmer Penguins dataset.

- **None:** All classes are assigned equal weight, such that $w_c = 1 \forall c \in C$
- **Balanced:** Class weights are inversely proportional to the class frequencies in the training data. The weight assigned to class c is given by $w_c = \frac{n}{K \cdot n_c}$, where n denotes the total number of training samples, K is the number of classes, and n_c is the number of samples belonging to class c .
- **Custom:** Users may specify their own class weights w_c for each class $c \in C$, allowing domain-specific adjustments based on prior knowledge or application requirements.

Figure 5 illustrates the effect of alternative class weighting schemes on the decision boundaries produced by SVMODT. In the first panel, equal weights are assigned to all classes. In the second panel, balanced weights are used, whereby class weights are set inversely proportional to the number of observations belonging to each class at the node. In the final panel, a custom weighting scheme is imposed for illustrative purposes, with weights specified as Adelie = 2, Gentoo = 1, and Chinstrap = 5.

Under balanced weighting, the resulting decision boundaries remain broadly similar to those obtained under equal weighting, although the separating regions shift modestly toward the centre of the feature space. This reflects the increased influence of relatively smaller classes during local SVM estimation. In contrast, the custom weighting specification produces substantially different decision regions. Owing to the large weight assigned to Chinstrap penguins, the model allocates a more expansive and pronounced classification region to that class. Adelie penguins form two detached decision regions, indicating a redistribution of boundary geometry in response to the weighting scheme. By comparison, the Gentoo decision region changes only marginally, consistent with its comparatively low assigned weight. These results demonstrate that node-specific class weighting provides a flexible mechanism for altering local decision boundaries and may be particularly useful when class imbalance or asymmetric misclassification costs are present.

Table 8 presents the macro-averaged F1 scores obtained under different class weighting strategies across ten benchmark datasets. The results indicate that class weighting has a notable impact on predictive performance across datasets. For datasets exhibiting multiple classes or pronounced class imbalance, such as Cardiotocography and Fertility, the choice of weighting strategy substantially affects the macro-averaged F1 score. Specifically, the Balanced weighting scheme frequently improves performance relative to the unweighted (None) configuration. Conversely, in datasets with more balanced class distributions, differences between weighting strategies are smaller, although the highest F1 scores are often achieved with either the Balanced or Custom weights. Overall, the results indicate that selecting an appropriate class weighting strategy is particularly important for datasets with multiple classes or significant class imbalance, while for more balanced datasets, weighting has a minimal impact.

Table 8: Comparison of macro-averaged F1 score (mean $\pm$ sd) across class weighting strategies. All SVMODT models are trained with `max_depth = 15`, `feature_method = "mutual"`, and adaptive `max_features`. *None*: `class_weights = "none"`; *Balanced*: `class_weights = "balanced"`. Bold values denote the highest F1 score for each dataset.

Dataset	N	X	L	Class Imbalance Ratio	STree	None	Balanced
WDBC Diagnosis	569	30	2	0.5938375	0.968 $\pm$ 0.004	0.943 $\pm$ 0.008	0.948 $\pm$ 0.008
Iris	150	4	3	1.0000000	0.965 $\pm$ 0.010	0.963 $\pm$ 0.006	0.945 $\pm$ 0.015
Echocardiogram	131	10	2	0.4886364	0.792 $\pm$ 0.009	0.800 $\pm$ 0.006	0.752 $\pm$ 0.040
Fertility	100	9	2	0.1363636	0.466 $\pm$ 0.003	0.468 $\pm$ 0.001	0.560 $\pm$ 0.041
Wine	178	13	3	0.6760563	0.959 $\pm$ 0.008	0.954 $\pm$ 0.010	0.952 $\pm$ 0.017
Cardiotocography-3	2126	21	3	0.1063444	0.812 $\pm$ 0.007	0.766 $\pm$ 0.017	0.827 $\pm$ 0.009
Cardiotocography-10	2126	21	10	0.0915371	0.716 $\pm$ 0.011	0.586 $\pm$ 0.071	0.746 $\pm$ 0.011
Ionosphere	351	33	2	0.5600000	0.879 $\pm$ 0.016	0.855 $\pm$ 0.021	0.858 $\pm$ 0.020
Dermatology	366	34	6	0.1785714	0.946 $\pm$ 0.013	0.956 $\pm$ 0.009	0.950 $\pm$ 0.015

Algorithm

Having outlined the principal components of SVMODT, we now present the complete training procedure. The preceding sections described each mechanism individually, including adaptive feature subsetting, feature selection strategies, regularisation controls, feature penalisation, and class weighting. Algorithm 4 integrates these components into a unified recursive tree-building framework.

Algorithm 4: SVMODT: SVM-based Oblique Decision Tree with Enhanced Splitting

Input: $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$: training dataset, d : current depth, $d_{\max}$: maximum depth, $n_{\min}$: minimum samples, m_{base} : base number of features, $\mathcal{C}$: set of all possible class labels, strat_m : feature selection strategy, strat_w : class weight strategy, $\text{method}_{\mathcal{F}}$: feature subset method, measure_I : impurity measure

Output: *tree*: leaf node or data split $\mathcal{D}^+, \mathcal{D}^-$

```

1 function SVMODT( $\mathcal{D}, d, d_{\max}, n_{\min}, m_{\text{base}}, \text{method}_{\mathcal{F}}, \text{measure}_I, \text{strat}_m, \text{strat}_w, \mathcal{F}_{\text{used}}, \lambda_{\text{pen}}, \tau_I, \mathcal{C}$ ):
2    $n \leftarrow |\mathcal{D}|$ 
3    $\mathcal{Y} \leftarrow \{y_i\}_{i=1}^n$  // Extract labels
4   // Stopping conditions
5   if  $d > d_{\max}$  or  $|\text{unique}(\mathcal{Y})| = 1$  or  $n < n_{\min}$  then
6     return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ )
7   // Dynamic feature selection and scaling
8    $m_d \leftarrow \text{calculate\_dynamic\_max\_features}(d, m_{\text{base}}, \text{strat}_m)$  HOW IS  $m_{\text{base}}$ 
9    $\mathcal{F}_d \leftarrow \text{select\_features\_with\_penalty}(\mathcal{D}, m_d, \text{method}_{\mathcal{F}}, \mathcal{F}_{\text{used}}, \lambda_{\text{pen}})$ 
10   $\mathbf{X}_{\text{scaled}}, \text{scaler} \leftarrow \text{scale\_node}(\mathcal{D}[\mathcal{F}_d])$ 
11   $\mathcal{F}_d \leftarrow \text{features}(\mathbf{X}_{\text{scaled}})$  // Update to non-constant features
12  if  $|\mathcal{F}_d| = 0$  then
13    return create_leaf_node(mode( $\mathcal{Y}_{\sqrt{\cdot}}$ ),  $n, \mathcal{C}$ ) // No valid features
14   $k \leftarrow |\text{unique}(\mathcal{Y}_{\sqrt{\cdot}})|$  // Number of classes at node
15   $I_p \leftarrow I(\mathcal{Y}, \text{measure}_I)$  // Parent impurity
16  // Binary vs Multiclass handling
17  if  $k = 2$  then
18    ( $\text{best\_model}, \mathcal{I}_L, \mathcal{I}_R$ )  $\leftarrow \text{BinarySplit}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w)$  WE HAD  $\text{strat}_m$  BEFORE
19    AND NOW WE HAVE  $\text{strat}_w$  // Refer to Algorithm 2
20  else
21    ( $\text{best\_model}, \mathcal{I}_L, \mathcal{I}_R, c_{\text{split}}$ )  $\leftarrow \text{MulticlassSplit}(\mathbf{X}_{\text{scaled}}, \mathcal{Y}, \text{strat}_w, I_{\text{parent}}, \text{measure}_I, \tau_I, n)$ 
22    // Refer to Algorithm 3
23  if  $\text{best\_model} = \text{null}$  or  $|\mathcal{I}_L| = 0$  or  $|\mathcal{I}_R| = 0$  then
24    return create_leaf_node(mode( $\mathcal{Y}$ ),  $n, \mathcal{C}$ )
25  // Handle small children
26  if  $|\mathcal{I}_L| < n_{\min}$  and  $|\mathcal{I}_R| < n_{\min}$  then
27    return create_leaf_node(mode( $\mathcal{Y}_{\sqrt{\cdot}}$ ),  $n, \mathcal{C}$ ) // Both children too small
28  // Update used features for penalty mechanism
29   $\mathcal{F}'_{\text{used}} \leftarrow \mathcal{F}_{\text{used}} \cup \mathcal{F}_d$ 
30  // Create internal node
31   $\text{node} \leftarrow \text{create\_internal\_node}(\text{best\_model}, \mathcal{F}_d, \text{scaler}, d, n)$ 
32  // Recursively build children
33  if  $|\mathcal{I}_L| \geq n_{\min}$  then
34     $\text{node.left} \leftarrow \text{SVMODT}(\mathcal{D}[\mathcal{I}_L], d + 1, d_{\max}, n_{\min}, m_{\text{base}}, \dots, \mathcal{F}'_{\text{used}}, \dots, \mathcal{C})$ 
35  else
36     $\text{node.left} \leftarrow \text{create\_leaf\_node}(\text{mode}(\mathcal{Y}[\mathcal{I}_L]), |\mathcal{I}_L|, \mathcal{C})$ 
37  if  $|\mathcal{I}_R| \geq n_{\min}$  then
38     $\text{node.right} \leftarrow \text{SVMODT}(\mathcal{D}[\mathcal{I}_R], d + 1, d_{\max}, n_{\min}, m_{\text{base}}, \dots, \mathcal{F}'_{\text{used}}, \dots, \mathcal{C})$ 
39  else
40     $\text{node.right} \leftarrow \text{create\_leaf\_node}(\text{mode}(\mathcal{Y}[\mathcal{I}_R]), |\mathcal{I}_R|, \mathcal{C})$ 
41  return node

```

Performance Evaluation of SVMODT with Hyperparameter Optimisation

We evaluate the predictive performance of SVMODT against a range of competing decision tree and support vector machine-based classification methods. These benchmarks include Classification and Regression Trees (CART) using [rpart](#), linear support vector machines using [e1071](#), logistic regression using [nnet](#), STree, and Accelerated Oblique Random Survival Forests using [aorsf](#). Collectively, these comparators provide a broad reference set spanning conventional axis-parallel trees, linear margin-based classifiers, and alternative oblique tree-based approaches.

To optimise the predictive performance of SVMODT, we first conducted a cross-validated grid

search over a predefined hyperparameter space for each benchmark dataset. Model selection was based on repeated 5-fold cross-validation across three independent random seeds. For each candidate configuration, the mean classification accuracy and its standard deviation across seeds were recorded. The final tuning configuration for each dataset was selected as the model with the highest mean accuracy, with ties resolved in favour of the lowest cross-seed standard deviation to prioritise stability. The selected hyperparameters were then re-evaluated using the remaining seeds under an identical 5-fold cross-validation design to obtain the reported out-of-sample performance of **SVMODT***.

The grid search tuned the principal structural and feature-selection components of **SVMODT**. These included: (i) the feature selection method (random, mutual, cor), corresponding respectively to random feature ranking, mutual information, and correlation-based screening; (ii) the class weighting scheme (none, balanced); (iii) the maximum feature strategy (constant, decrease, random), which determines how many predictors are available at each node; (iv) whether to apply feature penalisation (TRUE, FALSE); (v) the feature penalty weight (0.3, 0.6) when penalisation was active; (vi) the minimum impurity decrease threshold (0.001, 0.01, 0.1), used as a stopping and regularisation criterion; and (vii) the node-level maximum number of candidate features, chosen adaptively as either $\lfloor \sqrt{p} \rfloor$ or p , where p is the total number of predictors. For random feature strategies, the candidate proportion was sampled uniformly from the interval $[0.1, 0.9]$.

Competing methods were benchmarked using standard or literature-supported specifications. **STree*** denotes the Python implementation of **STree** using the dataset-specific kernel and regularisation parameters reported by the original authors (see Section 4.2; [Montañana et al. \(2025\)](#)). To ensure fair benchmarking, the **CART** baselines implemented via **rpart** were also tuned using cross-validated grid search over tree depth and complexity penalty under the same resampling framework. This tuning framework ensures that comparisons between **SVMODT** and competing classifiers reflect not only raw predictive performance, but also the benefits of principled hyperparameter optimisation under a consistent resampling protocol. Table 9 presents mean prediction accuracy across seven classifiers on nine benchmark datasets.

Overall competitiveness

SVMODT* remains strongly competitive across the benchmark suite, achieving the highest standalone accuracy on **Echocardiogram** (0.854) and matching the best result on **Iris** (0.971). In addition, it performs near the top tier on several other datasets, including **Wine** (0.970), **WDBC** (0.964), **Dermatology** (0.950), and **Cardiotocography-10** (0.795). These results are notable because **SVMODT*** relies exclusively on linear support vector machine splits at each node, whereas several competing methods including **STree*** and **AORSF** benefit from more flexible non-linear or ensemble-based decision boundaries.

Performance relative to **STree***

Despite this structural simplicity, **SVMODT*** remains competitive with **STree*** across most datasets. **STree*** attains the strongest results on **WDBC** (0.973), **Ionosphere** (0.947), and several multiclass tasks, but these gains often arise when non-linear kernels such as radial basis function (RBF) or polynomial kernels are used. By contrast, **SVMODT*** uses only linear node-wise hyperplanes, making the resulting tree substantially easier to interpret and requiring fewer hyperparameters to tune. On datasets where **STree*** uses a linear kernel, such as **Iris**, **Wine**, and **Dermatology**, the performance gap narrows considerably, and **SVMODT*** either matches or remains within a small margin of **STree***. This suggests that much of **STree***'s advantage is attributable to kernel flexibility rather than the tree induction procedure itself.

Performance relative to default **SVMODT**

Hyperparameter optimisation produces modest but meaningful gains on selected datasets. Improvements are observed on **Iris** (+0.005), **Echocardiogram** (+0.008), and **Cardiotocography-10** (+0.003) relative to the default specification. However, tuning does not uniformly improve performance: the default **SVMODT** remains stronger on **Wine** (+0.008), **Dermatology** (+0.022), and **Fertility** (+0.007). This indicates that the untuned configuration already provides a robust baseline, while more aggressive optimisation may overfit smaller or low-variance datasets. Accordingly, **SVMODT** appears relatively stable even without extensive parameter search.

Performance relative to tree-based baselines

Both SVMODT variants generally outperform conventional single-tree CART models on most datasets, particularly where oblique linear boundaries are advantageous. For example, on **WDBC**, **Wine**, **Iris**, and **Dermatology**, SVMODT substantially exceeds axis-aligned CART accuracy. However, on **Cardiotocography-3** and **Cardiotocography-10**, tuned CART remains highly competitive, suggesting that deep axis-parallel trees can still perform strongly when class structure aligns well with recursive univariate splits. Relative to ensemble tree methods, **AORSF** is the strongest overall competitor, achieving the best results on **Wine** (0.988) and **Dermatology** (0.976), while remaining highly competitive elsewhere. This is expected given the variance-reduction benefits of ensemble learning. Nevertheless, SVMODT offers a substantially more interpretable single-tree alternative while retaining competitive predictive performance.

Summary

Overall, the results demonstrate that SVMODT successfully bridges the gap between interpretability and predictive accuracy. It consistently outperforms conventional single decision trees, approaches the performance of more complex SVM-tree hybrids such as **STree**, and remains competitive with ensemble oblique forests on several datasets, all while preserving a transparent linear-split tree structure. SVMODT* demonstrates competitive predictive performance across the benchmark datasets, achieving the highest accuracy on several problems despite relying solely on linear node-wise hyperplanes. Hyperparameter optimisation yields modest but meaningful improvements on selected datasets, although the default configuration already serves as a strong and robust baseline, particularly for smaller or lower-dimensional problems where extensive tuning may overfit. Overall, the results suggest that oblique decision trees combining adaptive feature selection, feature diversity mechanisms, and controlled regularisation are especially effective for multiclass and high-dimensional classification tasks.

Table 9: Comparison of Mean Prediction Accuracy (mean $\pm$ sd) across classifiers. *STree** denotes Python STree with dataset-specific hyperparameters. *SVMODT** denotes SVMODT with tuned hyperparameters from grid search. Bold denotes the highest accuracy per dataset.

Dataset	N	X	L	CART*	SVM (linear)	AORSF	Log. Reg.	PPTree	STree*	SVMODT (Default)	SVMODT*
WDBC Diagnosis	569	30	2	0.925 $\pm$ 0.006	0.970 $\pm$ 0.004	0.971 $\pm$ 0.004	0.946 $\pm$ 0.007	0.966 $\pm$ 0.006	\textbf{0.973 $\pm$ 0.004}	0.969 $\pm$ 0.003	0.964 $\pm$ 0.006
Iris	150	4	3	0.931 $\pm$ 0.008	0.966 $\pm$ 0.011	0.955 $\pm$ 0.004	0.961 $\pm$ 0.011	\textbf{0.971 $\pm$ 0.005}	0.966 $\pm$ 0.011	0.966 $\pm$ 0.006	\textbf{0.971 $\pm$ 0.006}
Echocardiogram	131	10	2	0.853 $\pm$ 0.005	0.846 $\pm$ 0.004	0.809 $\pm$ 0.014	0.791 $\pm$ 0.018	0.831 $\pm$ 0.011	0.848 $\pm$ 0.016	0.846 $\pm$ 0.006	\textbf{0.854 $\pm$ 0.003}
Fertility	100	9	2	\textbf{0.881 $\pm$ 0.000}	0.874 $\pm$ 0.010	\textbf{0.881 $\pm$ 0.000}	0.840 $\pm$ 0.014	\textbf{0.881 $\pm$ 0.000}	\textbf{0.881 $\pm$ 0.000}	0.874 $\pm$ 0.010	\textbf{0.881 $\pm$ 0.000}
Wine	178	12	3	0.864 $\pm$ 0.025	0.961 $\pm$ 0.005	\textbf{0.988 $\pm$ 0.002}	0.983 $\pm$ 0.005	0.961 $\pm$ 0.014	0.964 $\pm$ 0.010	0.978 $\pm$ 0.008	0.970 $\pm$ 0.008
Cardiotography-3	2126	21	3	\textbf{0.919 $\pm$ 0.003}	0.897 $\pm$ 0.002	0.911 $\pm$ 0.002	0.892 $\pm$ 0.004	0.889 $\pm$ 0.003	0.903 $\pm$ 0.003	0.902 $\pm$ 0.004	0.902 $\pm$ 0.003
Cardiotography-10	2126	21	10	0.819 $\pm$ 0.006	0.820 $\pm$ 0.005	0.807 $\pm$ 0.003	\textbf{0.821 $\pm$ 0.005}	0.763 $\pm$ 0.006	0.808 $\pm$ 0.007	0.792 $\pm$ 0.004	0.795 $\pm$ 0.006
Ionosphere	351	33	2	0.887 $\pm$ 0.013	0.879 $\pm$ 0.005	0.921 $\pm$ 0.005	0.877 $\pm$ 0.012	0.859 $\pm$ 0.014	\textbf{0.947 $\pm$ 0.006}	0.896 $\pm$ 0.011	0.896 $\pm$ 0.008
Dermatology	366	34	6	0.929 $\pm$ 0.007	0.968 $\pm$ 0.006	\textbf{0.976 $\pm$ 0.005}	0.968 $\pm$ 0.010	0.905 $\pm$ 0.025	0.968 $\pm$ 0.006	0.972 $\pm$ 0.006	0.950 $\pm$ 0.005

4 Discussion

This paper has made two primary contributions to the R statistical computing ecosystem. First, we introduced **STreeR**, the first fully native R implementation of the STree algorithm (Montañana et al., 2021; Montañana et al., 2021), reconstructed without any Python dependency and validated against the original implementation across nine benchmark datasets. Second, we developed **SVMODT**, an extension of STree that incorporates adaptive feature subsetting, three embedded feature selection strategies (random, mutual information, and correlation), a feature penalisation mechanism for structural diversity, node-specific class weighting for imbalanced settings, and a multi-parameter regularisation framework. Together, these contributions make SVM-driven oblique decision trees fully accessible within the R ecosystem and substantially extend the methodological capabilities of the original framework.

The empirical results demonstrate that SVMODT is competitive with, and in several settings superior to, both axis-parallel trees and the Python STree benchmark despite restricting node classifiers to linear kernels. The linear constraint is deliberate: it ensures that each internal node produces an interpretable hyperplane expressible as a weighted sum of features, an advantage forfeited by kernel-based methods. Where STree* outperforms SVMODT*, the gap is largely attributable to non-linear kernels (RBF, polynomial) rather than the tree induction procedure itself, as the performance difference narrows substantially on datasets for which STree* also employs a linear kernel.

The feature penalisation mechanism, while producing modest improvements on average, demonstrates a clear structural effect: increasing λ_{pen} redistributes feature usage across nodes, promoting diversity in the learned decision boundaries. This behaviour mirrors the role of the `mtry` parameter in random forests, offering a single-tree analogue of the subspace-sampling heuristic that underpins ensemble diversity (Breiman et al., 1984). The adaptive maximum-feature strategies provide complementary control: the constant strategy delivers predictable computational cost, the decreasing strategy concentrates expressive capacity early in the tree where sample sizes are largest, and the random strategy introduces stochasticity that can reduce overfitting on smaller datasets.

The benchmark results also reveal a consistent pattern across regularisation parameters. Moderate values of n_{min} , d_{max} , and τ_I reliably improve or maintain out-of-sample accuracy relative to their extremes, and their effects are largely orthogonal: minimum node size governs local sample adequacy, maximum depth constrains global tree growth, and the impurity threshold controls split quality. Practitioners can therefore treat them as independent controls, tuning each in turn under a standard cross-validation protocol.

Recent work on the Oblique Decision Tree Ensemble (ODTE) (Montañana et al., 2025) extends the STree framework to ensembles by placing one-vs-one or one-vs-rest SVMs at each split and dynamically selecting the classifier that minimises class impurity. ODTE addresses scalability and class-imbalance limitations inherent to single-tree SVM approaches. The results reported by Montañana et al. (2025) suggest that ensemble aggregation can close a substantial portion of the accuracy gap between single-tree and deep-network classifiers, while preserving the interpretive structure of decision rules. The success of ODTE provides a natural motivation for extending SVMODT in the ensemble direction, a direction we discuss further below.

Two recurring themes emerge from this line of research: (i) leveraging multivariate oblique boundaries to capture complex class geometry that axis-parallel splits approximate only with excessive depth, and (ii) combining transformation or ensemble strategies to improve diversity and robustness without sacrificing the transparency of individual decision rules. SVMODT addresses the first theme through its linear SVM node classifiers; the second remains an open avenue for future work.

5 Future Work

The most immediate natural extension to SVMODT tree is an oblique forest built on SVMODT nodes, analogous to the ODTE of Montañana et al. (2025) but with the feature selection, penalisation, and class-weighting mechanisms developed here. A bagged or boosted ensemble of SVMODT trees would be expected to reduce variance substantially on high-dimensional datasets while preserving the interpretability of individual tree paths through feature importance summaries at the node level. The `feature_method` and `penalize_used_features` arguments developed for SVMODT provide a natural mechanism for controlling intra-ensemble diversity.

6 Limitations

Several limitations of the current implementation and experimental design should be acknowledged.

Computational cost of hyperparameter tuning. The grid search conducted for SVMODT* involved repeated 5-fold cross-validation across multiple seeds over a parameter space spanning feature selection method, class weighting, maximum feature strategy, penalisation, and impurity threshold. For the larger datasets (Cardiotography, $n = 2126$), this is computationally demanding. Unlike random forests, SVMODT does not currently support parallel tree construction, and each node requires solving a quadratic equation whose cost scales quadratically with the number of local training samples. Future work should explore parallelisation via [parallel](#) or [foreach](#).

Restriction to linear kernels. The decision to restrict node classifiers to linear SVMs improves interpretability and avoids kernel hyperparameter selection, but forfeits representational capacity in settings where class boundaries are strongly non-linear. On Ionosphere and several multiclass tasks, STree* achieves meaningfully higher accuracy using RBF or polynomial kernels. Extending SVMODT to support non-linear kernels as an optional configuration alongside tools for extracting and visualising the resulting boundaries would broaden its applicability without sacrificing the linear default for users who prioritise transparency.

Benchmark scope. The experimental evaluation covers nine moderate-sized tabular datasets ranging from $n = 100$ to $n = 2126$ observations and up to 34 features. Performance on ultra-high-dimensional data (e.g., genomics or text), large-scale datasets ($n \gg 10^4$), sparse feature matrices, or datasets with mixed variable types (ordinal, nominal, continuous) has not been characterised.

7 Conclusion

We introduced the [svmodt](#) package, providing the R community with two complementary tools for SVM-based oblique decision trees. STreeR offers a fully native R reimplement of the STree algorithm, validated to match the accuracy of the original Python implementation while operating independently within the R ecosystem. SVMODT extends this foundation with adaptive feature selection, feature penalisation, node-specific class weighting, and a multi-parameter regularisation framework, collectively enabling practitioners to balance predictive performance, structural diversity, and model complexity in a principled way.

Across nine benchmark datasets, SVMODT with tuned hyperparameters (SVMODT*) consistently outperforms axis-aligned CART, matches or approaches linear SVM performance, and remains competitive with the more flexible STree* implementation, all while relying solely on linear node-wise hyperplanes that preserve the transparency of each decision rule. The default (untuned) SVMODT configuration already provides a strong baseline, and the package's design encourages systematic exploration through its compatibility with standard cross-validation workflows in [caret](#) and [rsample](#).

The package is freely available on GitHub and is under active development. We view the ensemble extension, large-scale solver integration, and regression support as the most important directions for subsequent releases. We hope that [svmodt](#) serves as a productive foundation for both applied practitioners seeking interpretable oblique classifiers and researchers exploring the theoretical and algorithmic frontiers of SVM-based decision tree learning.

8 Acknowledgements

The authors would like to thank Professor **Natalia Da Silva** and Professor **Dianne Helen Cook** for their constructive discussions and valuable insights regarding oblique decision trees and their implementations in R. The authors also acknowledge the use of OpenAI's GPT-5 model (ChatGPT) for editorial assistance and technical writing support during the preparation of this manuscript. All conceptual development, analysis, data processing, coding, and interpretation of results were performed independently by the author. The AI tool was used solely to refine language clarity, improve structure, and ensure stylistic consistency in the documentation.

9 Appendix

Algorithm 5: STree (Multi-class SVM-based oblique decision tree)

Input: $\mathcal{D}' = \{(x_i, y_i)\}_{i=1}^{t'}$: Data
Output: *tree*: the root node of an SVM-based oblique decision tree

```

1 function STree( $\mathcal{D}'$ ):
2   if stopping_condition( $\{y_i\}_{i=1}^t$ ) then
3     return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))           // Leaf node
4    $k' \leftarrow \text{num\_different\_labels}(\{y_i\}_{i=1}^t)$ 
5    $r \leftarrow \frac{k'(k'-1)}{2}$ 
6    $\mathcal{Y}' \leftarrow \{y'_1, \dots, y'_{k'}\}$            // set of labels
7    $I_{all} \leftarrow I(\{y_i\}_{i=1}^t)$            //  $I(\cdot)$  is an information theory measure
8   if (OvO) then
9      $n_{models} \leftarrow r$ 
10  else
11     $n_{models} \leftarrow k'$            // OvR
12  for  $j = 1$  to  $n_{models}$  do
13    if ( $j = k' = 2$ ) then
14      break           // Two labels, one SVM is enough
15    if (OvO) then
16      Let  $c_a$  and  $c_b$  the pair of labels corresponding to index  $j$ 
17       $models[j] \leftarrow \text{SVM}(\mathcal{D}'^{\downarrow c_a} \cup \mathcal{D}'^{\downarrow c_b})$ 
18    else
19       $models[j] \leftarrow \text{SVM}(\mathcal{D}')$  using binary class  $y'_j$  (+) vs rest (-)           // OvR
20   $\mathcal{D}^+, \mathcal{D}^- \leftarrow models[j](\vec{x}_j) \quad \forall (\vec{x}) \in \mathcal{D}'$ 
21   $I_j \leftarrow \frac{|\mathcal{D}^+|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^+|}) + \frac{|\mathcal{D}^-|}{|\mathcal{D}'|} I(\{y_i\}_{i=1}^{|\mathcal{D}^-|})$ 
22   $IG_j \leftarrow I_{all} - I_j$ 
23   $b^* = \arg \max_{j=1, \dots, n_{models}} IG_j$ 
24  if  $IG_{b^*} > 0$  then
25     $node \leftarrow \text{create\_node}(models[b^*])$ 
26     $node.left \leftarrow \text{STree}(\mathcal{D}^+)$ 
27     $node.right \leftarrow \text{STree}(\mathcal{D}^-)$ 
28  else
29    return create_leaf_node(mode( $\{y_i\}_{i=1}^t$ ))           // No split gain
30  return node

```

dataset	feature_method	class_weights	max_features_strategy	penalize_used_features	feature_penalty_weight	min_impurity_decrease	max_features
wdbc	mutual	none	random	FALSE	0.3	0.100	NA
iris	cor	none	decrease	FALSE	0.3	0.001	2
echocardiogram	cor	none	constant	FALSE	0.3	0.001	3
fertility	cor	none	constant	FALSE	0.3	0.001	3
wine	cor	none	constant	FALSE	0.3	0.001	13
ctg3	cor	none	constant	FALSE	0.3	0.001	21
ctg10	mutual	balanced	random	TRUE	0.3	0.010	NA
ionosphere	random	none	decrease	FALSE	0.3	0.001	33
dermatology	random	balanced	constant	FALSE	0.3	0.100	5

Bibliography

- M. Bala and R. Agrawal. Optimal decision tree based multi-class support vector machine. *Informatica*, 35(2), 2011. [p1, 4]
- K. Bennett and J. Blue. A support vector machine approach to decision trees. In *1998 IEEE International Joint Conference on Neural Networks Proceedings. IEEE World Congress on Computational Intelligence (Cat. No.98CH36227)*, volume 3, pages 2396–2401 vol.3, 1998. doi: 10.1109/IJCNN.1998.687237. [p1, 4]
- L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, 1984. URL <http://lyle.smu.edu/~mhd/8331f06/cart.pdf>. [p1, 21]
- C. E. Brodley and P. E. Utgoff. Multivariate decision trees. *Machine Learning*, 19:45–77, 1995. URL <https://api.semanticscholar.org/CorpusID:16836572>. [p2, 3]
- S. Bulò and P. Kotschieder. Neural decision forests for semantic image labelling. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 81–88, 2014. doi: 10.1109/CVPR.2014.18. [p3]
- L. Cañete, R. Monroy, and M. Medina-Pérez. A review and experimental comparison of multivariate decision trees. *IEEE Access*, PP:1–1, 08 2021. doi: 10.1109/ACCESS.2021.3102239. [p1]
- J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*, 408:189–215, 2020. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2019.10.118>. URL <https://www.sciencedirect.com/science/article/pii/S0925231220307153>. [p4]
- M. Chabbouh, S. Bechikh, E. Mezura-Montes, and L. Ben Said. Evolutionary optimization of the area under precision-recall curve for classifying imbalanced multi-class data. *Journal of Heuristics*, 31, 01 2025. doi: 10.1007/s10732-024-09544-z. [p3]
- C. Cortes and V. Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995. [p1, 4]
- N. Cristianini. An introduction to support vector machines and other kernel-based learning methods, 2000. [p1, 4]
- N. da Silva, D. Cook, and E.-K. Lee. A projection pursuit forest algorithm for supervised classification. *Journal of Computational and Graphical Statistics*, 30(4):1168–1180, 2021. doi: 10.1080/10618600.2020.1870480. URL <https://doi.org/10.1080/10618600.2020.1870480>. [p3]
- J.-x. Dong, A. Krzyzak, and C. Y. Suen. Fast svm training algorithm with decomposition on very large data sets. *IEEE transactions on pattern analysis and machine intelligence*, 27(4):603–618, 2005. [p4]
- M. R. Frea. *Small nets and short paths: Optimising neural computation*. PhD thesis, 1990. [p3]
- M. Friedl and C. Brodley. Decision tree classification of land cover from remotely sensed data. *Remote Sensing of Environment*, 61(3):399–409, 1997. ISSN 0034-4257. doi: [https://doi.org/10.1016/S0034-4257\(97\)00049-7](https://doi.org/10.1016/S0034-4257(97)00049-7). URL <https://www.sciencedirect.com/science/article/pii/S0034425797000497>. [p3]
- J. H. Friedman and J. W. Tukey. A projection pursuit algorithm for exploratory data analysis. *IEEE Transactions on computers*, 100(9):881–890, 2006. [p3]
- S. I. Gallant. Optimal linear discriminants. *Eighth International Conference on Pattern Recognition*, pages 849–852, 1986. URL <https://cir.nii.ac.jp/crid/1573668924476144256>. [p3]
- M. Ganaie, M. Tanveer, P. Suganthan, and V. Snasel. Oblique and rotation double random forest. *Neural Networks*, 153:496–517, 2022. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2022.06.012>. URL <https://www.sciencedirect.com/science/article/pii/S0893608022002258>. [p1]
- C.-W. Hsu and C.-J. Lin. A comparison of methods for multiclass support vector machines. *IEEE transactions on Neural Networks*, 13(2):415–425, 2002. [p4]
- J. Kittler. Feature selection and extraction. *Handbook of Pattern Recognition and Image Processing*, pages 59–83, 1986. URL <https://cir.nii.ac.jp/crid/1573950400671608448>. [p3]
- M. Koziol and M. Wozniak. *Multivariate Decision Trees vs. Univariate Ones*, pages 275–284. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. ISBN 978-3-540-93905-4. doi: 10.1007/978-3-540-93905-4_33. URL https://doi.org/10.1007/978-3-540-93905-4_33. [p3]

- J. B. Kruskal. Toward a practical method which helps uncover the structure of a set of multivariate observations by finding the linear transformation which optimizes a new “index of condensation”. In *Statistical computation*, pages 427–440. Elsevier, 1969. [p3]
- Y. Lee, D. Cook, J. Park, and E.-K. Lee. Pptree: Projection pursuit classification tree. *Electronic Journal of Statistics*, 7(1):1369 – 1386, 2013. ISSN 1935-7524. doi: 10.1214/13-EJS810. [p3]
- T. D. Lemmond, B. Y. Chen, A. O. Hatch, and W. G. Hanley. *An Extended Study of the Discriminant Random Forest*, pages 123–146. Springer US, Boston, MA, 2010. ISBN 978-1-4419-1280-0. doi: 10.1007/978-1-4419-1280-0_6. URL https://doi.org/10.1007/978-1-4419-1280-0_6. [p3]
- H. Liu and R. Setiono. Feature transformation and multivariate decision tree induction. In S. Arikawa and H. Motoda, editors, *Discovery Science*, pages 279–291, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. ISBN 978-3-540-49292-4. [p3]
- V. Menkovski, I. T. Christou, and S. Efremidis. Oblique decision trees using embedded support vector machines in classifier ensembles. In *2008 7th IEEE International Conference on Cybernetic Intelligent Systems*, pages 1–6, 2008. doi: 10.1109/UKRICIS.2008.4798937. [p1]
- B. H. Menze, B. M. Kelm, D. N. Splitthoff, U. Koethe, and F. A. Hamprecht. On oblique random forests. In D. Gunopulos, T. Hofmann, D. Malerba, and M. Vazirgiannis, editors, *Machine Learning and Knowledge Discovery in Databases*, pages 453–469, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-23783-6. [p3]
- J. Mingers. An empirical comparison of pruning methods for decision tree induction. *Machine Learning*, 4:227–243, 01 1989. doi: 10.1023/A:1022604100933. [p3]
- R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: A single multi-class oblique decision tree based on support vector machines. In E. Alba, G. Luque, F. Chicano, C. Cotta, D. Camacho, M. Ojeda-Aciego, S. Montes, A. Troncoso, J. Riquelme, and R. Gil-Merino, editors, *Advances in Artificial Intelligence*, pages 54–64, Cham, 2021. Springer International Publishing. ISBN 978-3-030-85713-4. [p4, 21]
- R. Montañana, J. A. Gámez, and J. M. Puerta. Stree: a single multi-class oblique decision tree based on support vector machines, 2021. [p1, 4, 21]
- R. Montañana, J. A. Gámez, and J. M. Puerta. Ode—an ensemble of multi-class svm-based oblique decision trees. *Expert Systems with Applications*, 273:126833, 2025. ISSN 0957-4174. doi: <https://doi.org/10.1016/j.eswa.2025.126833>. URL <https://www.sciencedirect.com/science/article/pii/S0957417425004555>. [p18, 21]
- M. Nanda, K. Seminar, D. Nandika, and A. Maddu. A comparison study of kernel functions in the support vector machine and a comparison study of kernel functions in the support vector machine and its application for termite detection, 2018. [p4]
- G. Pagallo and D. Haussler. Boolean feature discovery in empirical learning. *Machine Learning*, 5:71–99, 1990. URL <https://api.semanticscholar.org/CorpusID:5661437>. [p1, 3]
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. [p1]
- J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1986. URL <https://api.semanticscholar.org/CorpusID:189902138>. [p3]
- J. R. Quinlan. *C4. 5: programs for machine learning*. Elsevier, 2014. [p1, 3]
- E. Reynolds, B. Callaghan, and M. Banerjee. Svm–cart for disease classification. *Journal of Applied Statistics*, 46(16):2987–3007, 2019. doi: 10.1080/02664763.2019.1625876. URL <https://doi.org/10.1080/02664763.2019.1625876>. PMID: 33012942. [p4]
- C. Schaffer. Deconstructing the digit recognition problem. In D. Sleeman and P. Edwards, editors, *Machine Learning Proceedings 1992*, pages 394–399. Morgan Kaufmann, San Francisco (CA), 1992. ISBN 978-1-55860-247-2. doi: <https://doi.org/10.1016/B978-1-55860-247-2.50056-5>. URL <https://www.sciencedirect.com/science/article/pii/B9781558602472500565>. [p3]
- F. Takahashi and S. Abe. Decision-tree-based multiclass support vector machines. volume 3, pages 1418 – 1422 vol.3, 12 2002. ISBN 981-04-7524-1. doi: 10.1109/ICONIP.2002.1202854. [p1, 4]

- A. K. Y. Truong. Fast growing and interpretable oblique trees via logistic regression models. 2009. URL <https://api.semanticscholar.org/CorpusID:57884720>. [p3]
- P. C. Young. Recursive estimation and time-series analysis: An introduction. 1984. URL <https://api.semanticscholar.org/CorpusID:60181335>. [p3]
- L. Zhang and P. N. Suganthan. Oblique decision tree ensemble via multisurface proximal support vector machine. *IEEE Transactions on Cybernetics*, 45(10):2165–2176, 2015. doi: 10.1109/TCYB.2014.2366468. [p3]

Aneesh Agarwal
Monash University

aaga0022@student.monash.edu

Jack Jewson
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Jack.Jewson@monash.edu

Erik Sverdrup
Monash University
Department of Econometrics and Business Statistics, Monash University, Australia
Erik.Sverdrup@monash.edu